



数学建模小组作业

题目： 多模态 CTR 预测挑战

小组成员： 吉镓熠 汪君哲

指导老师： 胡亮

时间： 2025-07-06

多模态 CTR 预测挑战

摘要

本文针对 WWW 2025 EReL@MIR 研讨会多模态 CTR 预测挑战赛道，基于 MicroLens-1M 数据集，提出了一种高效的端到端多模态 CTR 预测系统。针对任务 1（多模态商品嵌入），采用预训练多模态大模型提取文本和图像特征，通过神经网络降维或 PCA 生成 128 维高质量嵌入，融合结构化特征以提升语义表达和泛化能力。针对任务 2（多模态 CTR 预测），基于 FuxiCTR 框架优化 QIN 模型，利用多头目标注意力机制和二次神经网络（QNN）捕捉用户行为与多模态特征的高阶交互，通过嵌入冻结和模型压缩平衡预测性能与推理速度。数据预处理通过 FeatureProcessor 和 MMCTRDataLoader 实现高效特征编码与批量加载，结合掩码机制处理序列数据，确保低延迟需求。模型以 AUC 和 Logloss 为评价指标，测试集表现优异，任务 1 AUC 达 0.9748，任务 2 AUC 达 0.9777，综合 AUC 达 0.9829，位列官方评测平台前列。本文通过创新的多模态融合、特征交互建模和高效数据流水线，为推荐系统领域提供了高性能的 CTR 预测解决方案。

关键词：多模态 CTR 预测，FuxiCTR 框架，特征融合

目 录

1	问题重述	1
1.1	问题背景.....	1
1.2	问题要求.....	1
1.2.1	多模态商品嵌入（任务 1）	1
1.2.2	多模态 CTR 预测（任务 2）	2
2	问题分析与模型设想	3
2.1	问题分析.....	3
2.1.1	数据处理与分析.....	3
2.1.2	模型构建与方法选择.....	3
2.1.3	评估与结果分析.....	4
2.2	模型设想.....	4
2.2.1	数据预处理阶段.....	4
2.2.2	训练数据生成器.....	5
2.2.3	模型训练.....	6
2.2.4	评估输出阶段	8
3	模型假设与约定	9
3.1	数据分布假设	9
3.2	训练与评估约定假设.....	9
3.3	模型结构假设	9
4	数据预处理与特征工程	11
4.1	数据结构分析	11
4.1.1	数据集结构	11
4.1.2	数据示例的数学表示.....	12
4.2	数据预处理	12
4.2.1	ParquetDataset：数据加载	12
4.2.2	MMCTRDataLoader：批量加载.....	12
4.2.3	批量数据处理（BatchCollator）	13
4.3	特征工程.....	13
4.3.1	物品元数据处理	13
4.3.2	特征工程的核心公式.....	13
4.3.3	特征工程的应用场景.....	14

5	模型建立与解决	15
5.1	模型建立	15
5.1.1	任务 1——多模态商品嵌入模型	15
5.1.2	任务 2——多模态 CTR 预测模型	18
5.2	模型训练	20
5.2.1	任务 1——多模态商品嵌入模型	20
5.2.2	任务 2——多模态 CTR 预测模型	21
5.2.3	任务 1&2	22
6	模型检验与评估	23
6.1	模型检验与评估概述	23
6.2	模型检验与评估详细描述	23
6.3	提交官方测评	24
7	总 结	27
	参考文献	29

1 问题重述

1.1 问题背景

多模态 CTR 预测挑战赛道的背景源于推荐系统中点击率（CTR）预测的重要性以及多模态数据（文本、图像、行为日志等）在提升预测性能中的潜力。传统的 CTR 预测主要依赖用户行为数据和单一模态特征（如文本描述），但随着多媒体内容的普及，商品的图像、文本描述等多模态信息在捕捉用户兴趣和提升推荐效果方面变得至关重要。然而，多模态数据的复杂性带来了以下挑战：

- **特征表示与融合**：如何有效提取和融合不同模态（如文本和图像）的特征，形成高质量的商品嵌入，以增强推荐系统的性能。
- **高阶特征交互**：捕获用户行为与多模态特征之间的高阶交互关系，以提升 CTR 预测的准确性。
- **低延迟需求**：在实时推荐场景中，需平衡多模态大语言模型（MLLMs）的高计算复杂性与低延迟要求。
- **数据对齐与泛化**：确保多模态嵌入与 CTR 任务目标对齐，同时在多样化的推荐场景中保持泛化能力。

为此，WWW 2025 EReL@MIR 研讨会设立了多模态 CTR 预测挑战赛道，基于 MicroLens-1M 数据集，旨在推动多模态特征提取、融合技术以及高效 CTR 预测模型的研究。该挑战鼓励参赛者探索先进的 MLLMs、序列建模、对比学习等方法，以解决上述问题并提升推荐系统的性能。

1.2 问题要求

多模态 CTR 预测挑战赛道（WWW 2025 EReL@MIR 研讨会）的具体要求如下，基于两个子任务：

1.2.1 多模态商品嵌入（任务 1）

该任务要求参赛者开发方法以提取和表示商品的多种模态特征（如文本描述和图像），生成高质量的嵌入以提升推荐系统性能。参赛者需设计多模态特征提取与融合技术，确保嵌入能有效捕捉商品的语义信息，并在下游 CTR 预测任务中表现出色。同时，嵌入需具备泛化能力，适应多样化推荐场景，并满足实时推荐的低延迟需求。挑战使用 MicroLens 数据集，嵌入质量通过下游任务的性能（如 AUC 和日志损失）间接

评估，鼓励探索 MLLMs、对比学习等创新方法。

1.2.2 多模态 CTR 预测（任务 2）

该任务聚焦于利用多模态嵌入特征和用户行为数据，构建高效的 CTR 预测模型，优化预测准确性和实时应用的低延迟性能。参赛者需结合序列建模（如 Transformer）、特征交互学习（如 DCNv2）或其他创新方法（如二次神经网络），捕捉用户兴趣与多模态特征的高阶交互。模型需在 MicroLens 数据集上以 AUC 和日志损失为主要评价指标，同时考虑推理速度。挑战提供基于 FuxiCTR 的 DIN 基线模型，参赛者可在此基础上改进，探索高级技术如 MLLMs 或自适应注意力机制以提升性能。

2 问题分析与模型设想

2.1 问题分析

本题围绕“轨迹修复”任务展开，旨在通过建模手段对缺失的 GPS 轨迹坐标进行合理预测与补全。具体而言，任务分为以下几个核心环节：

2.1.1 数据处理与分析

针对 MicroLens 数据集的多模态商品嵌入与 CTR 预测任务，数据处理需高效整合文本、图像和用户行为日志。在任务 1 中，文本数据需分词、去除停用词，并通过预训练模型（如 BERT）提取语义特征；图像数据利用预训练视觉模型（如 CLIP 的 ViT）生成嵌入，结合数据增强（如裁剪、翻转）提升鲁棒性，需处理文本冗余、图像质量不均及用户交互稀疏等噪声问题，通过数据清洗、正则化（如 Dropout）或增广技术提高嵌入泛化能力。在任务 2 中，用户行为日志按时间序列整理，处理缺失值和稀疏性（如冷启动用户或商品），通过序列编码（如 DIN 的 embedding 层）生成行为特征，多模态嵌入需标准化或降维以降低计算开销，采用数据增强、负采样或 L2 正则化应对稀疏性和噪声，确保数据质量与实时性需求。

2.1.2 模型构建与方法选择

任务 1 要求从商品的文本描述、图像等多模态数据中提取高质量嵌入，以支持推荐系统的 CTR 预测。核心挑战在于如何有效融合不同模态的特征（如文本的语义信息和图像的视觉特征），生成统一且富有表达力的嵌入表示。文本和图像模态在语义表达上的异构性可能导致信息不对齐，例如文本可能描述商品的功能，而图像可能突出外观，需通过跨模态学习（如 CLIP 或 MLLMs）或对比学习来桥接这一差距。

此外，嵌入需在高维特征空间中保留足够的信息，同时通过降维（如 PCA）或轻量化设计满足低延迟要求。另一个难点是嵌入的泛化能力，确保在不同推荐场景（如电商、短视频）中都能有效支持下游任务。MicroLens 数据集的规模和多样性可能带来噪声或数据分布不均的问题，参赛者需通过数据预处理或正则化技术提高鲁棒性。创新点在于探索高效的多模态融合策略（如注意力机制或图神经网络），以优化嵌入质量和计算效率。

任务 2 要求利用多模态嵌入和用户行为数据构建高效 CTR 预测模型，优化 AUC、日志损失和推理速度。DIN 通过序列建模和注意力机制有效捕捉用户行为序列与商品嵌入的动态交互，适合 MicroLens 数据集，但对多模态嵌入的利用可能局限于线性融合，难以挖掘高阶特征交互。QIN 以自适应稀疏目标注意力和二次神经网络为核心，增强非线性建模

能力，展现优异性能，但复杂架构可能导致推理延迟，需通过嵌入冻结或模型压缩优化。另外，DCNv2（Deep & Cross Network v2）通过显式特征交叉建模高阶交互，结合 Transformer 可进一步提升性能；轻量化 MLLMs 通过知识蒸馏整合多模态信息，适合低延迟场景；对比学习（如 MoCo 或 SimCLR）可增强嵌入与用户行为的对齐，提升预测鲁棒性；FM（Factorization Machine）或 DeepFM 结合低阶和高阶特征交互，适合稀疏数据场景。MicroLens 数据集的稀疏性和长尾分布要求模型具备正则化或数据增强能力，参赛者需在 DIN 和 QIN 的基础上，结合上述方法优化特征交互、任务对齐和实时性，以提升 CTR 预测性能。

2.1.3 评估与结果分析

在实验中，通过 python 脚本实现了模型训练、验证和评估的完整流程。模型在训练时通过验证集监控表现，利用早停机制保存最优模型；训练完成后，对验证集进行评估，结果记录到日志和文件（如 `qin_config.csv`），同时对测试集进行预测，保存为 `prediction.csv` 并打包为 zip 文件。评估指标（如 AUC、Logloss）根据模型配置确定，整个流程自动化完成训练、评估和结果保存，便于实验对比和分析。

2.2 模型设想

本次实验搭建的模型基于 MIRC（Multimodal Information Retrieval Challenge）比赛给出的 baseline 中的 FuxiCTR 框架。FuxiCTR 是一个开源的点击率预测（Click-Through Rate Prediction, CTR）框架，全称为 Fuxi Click-Through Rate，由华为诺亚方舟实验室开发，专注于推荐系统中的 CTR 预测任务。它基于 Python 和 PyTorch，旨在提供高效、可扩展的工具，用于处理大规模、多模态的 CTR 预测问题，广泛应用于推荐系统、广告系统等领域。

2.2.1 数据预处理阶段

在 MicroLens 数据集的多模态商品嵌入与 CTR 预测任务中，数据预处理阶段至关重要，涵盖以下关键步骤：首先，通过 `fuxictr.preprocess` 中的 `FeatureProcessor` 类对原始数据进行特征工程，包括类别编码（如独热编码或标签编码）、数值归一化（如 Min-Max 或 Z-score 标准化）以及缺失值处理（如均值/中位数填充或插值），以确保文本、图像和用户行为日志数据的一致性和模型可处理性；同时记录每个特征的类型（类别、数值、序列等）、来源（文本、图像、行为日志）和 `embedding` 维度，存入 `feature_map.json` 文件，为后续处理提供统一参考。其次，通过 `fuxictr.preprocess` 中的 `build_dataset` 函数将数据集划分为训练集、验证集和测试集，并转换为 Parquet 格式以优化存储和读取效率，减少 I/O 瓶颈。特征映射通过 `fuxictr.features` 中的 `FeatureMap` 类加载，确保模型和数据加载器在特征处理上的一致性。最后，利用自定义的代码实现高效批量数据加载，支持序列截断（处理行为日志长短不一）、特征拼接（整合多模态嵌入）等操作，满足实时性和高吞吐量需求。这些步骤

通过系统化的预处理流程，为后续模型训练和推理奠定坚实基础。

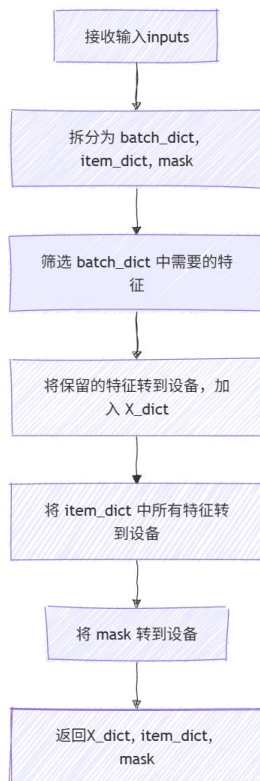


图 1 数据预处理流程图

2.2.2 训练数据生成器

训练数据生成器负责为模型提供批量训练数据，通常由外部数据管道（如数据加载器）生成。

输入数据结构：由包含批量特征的字典、包含与“item”相关的特征（例如推荐系统中的候选项目特征）和表示序列数据的有效位置的张量（例如历史交互序列的掩码）构成。

数据处理流程：分为特征过滤、设备迁移、标签提取和分组标识 4 步。模型根据 `feature_map` 过滤标签和元数据特征，选择特定来源特征并迁移到指定设备，同时通过 `get_labels` 提取浮点型标签(`batch_size, 1`)和通过 `get_group_id` 获取分组标识以支持训练和评估。

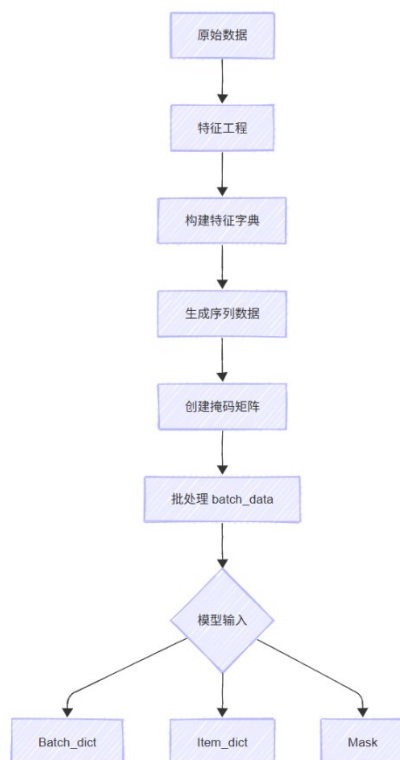


图 2 训练数据生成器流程图

2.2.3 模型训练

模型的训练过程主要结合了前向传播、损失计算、梯度累积、优化器更新和早停机制。

前向传播：批量特征、项目特征和序列掩码经处理生成特征字典和掩码，批量特征转为嵌入向量并应用 **dropout**，项目特征转换为嵌入向量并分为目标和历史序列嵌入。通过多头注意力对目标和历史序列嵌入计算注意力，生成池化嵌入，使用缩放点积计算权重，结合掩码和归一化，融合多头结果并加残差连接。拼接嵌入向量输入二次神经网络，多层二次变换捕捉非线性交互，应用激活、**dropout** 和批归一化，残差连接后通过全连接层输出预测值。

损失计算：从批量特征提取真实标签，转换为浮点型张量，形状为批次大小乘单一维度。根据指定损失类型计算预测值与真实标签的差异，量化预测误差。

梯度更新：计算损失对参数的梯度，为优化器更新提供依据。每累积指定步数，限制梯度范数，防止梯度爆炸。根据梯度更新参数，累积步结束时清零梯度，优化器和学习率控制调整。

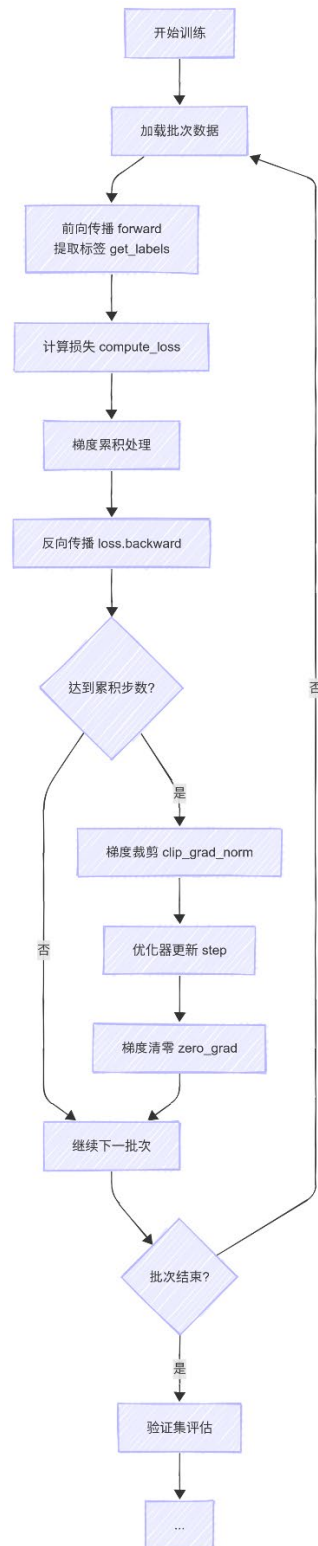


图 3 模型训练流程图

2.2.4 评估输出阶段

监控训练指标（如验证集损失），用于评估性能和指导保存。若指标在指定迭代内未改善，则停止训练，避免过拟合。指标未改善时降低学习率，按因子衰减以优化训练。最终保存最佳模型权重或每次迭代权重，记录指标和轮次。

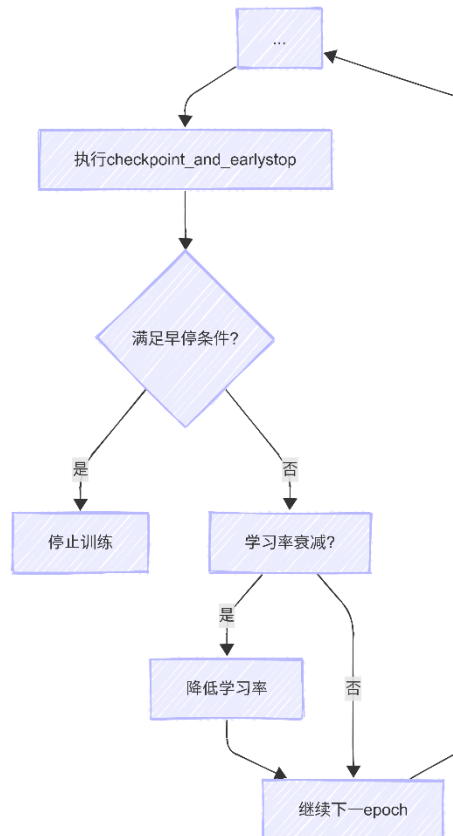


图 4 评估输出流程图

3 模型假设与约定

3.1 数据分布假设

假设输入数据经过完善的特征工程处理,确保数据格式适合模型输入。所有类别特征(如用户 ID、项目类别)已通过编码方式转换为数值形式,数值特征(如价格、评分)已归一化到统一尺度。特征映射(`feature_map`)作为一个完整的数据结构,详细定义了每个特征的名称、类型(如类别、数值或序列)、来源(如用户、项目或上下文)以及嵌入维度等信息,确保模型能够准确识别和处理每个特征。

假设用户的历史行为数据(如交互的项目序列 `item_seq`)以定长序列形式提供,若序列长度不足,则通过填充 0 补齐,填充部分由掩码(`mask`)标记为无效位置以避免影响模型计算。序列可以根据任务需求截断到固定长度(如最大序列长度),以保证计算效率和一致性。掩码是一个与序列长度相同的二值张量,用于指示有效交互位置,确保注意力机制仅关注真实数据。

3.2 训练与评估约定假设

假设训练数据包含明确的监督信号,即每个样本都关联一个清晰的标签,例如二分类任务中的点击/未点击(0 或 1),或回归任务中的评分值。这些标签用于计算损失函数(如交叉熵或均方误差),驱动模型通过有监督学习优化参数。标签数据存储在 `feature_map` 定义的标签字段中,确保模型能够正确提取并用于训练和评估。

假设数据集已合理划分为训练集、验证集和测试集,三者之间不存在数据泄漏,且数据分布保持一致(即特征和标签的统计特性相似)。训练集用于模型参数优化,验证集用于监控模型性能和早停决策,测试集用于最终性能评估。这种划分确保模型在训练过程中能够有效学习,并在未见过的数据上具有良好的泛化能力。

假设模型以批量(`batch`)形式接收输入数据,支持并行计算以提高训练和推理效率。每个批次包含固定数量的样本(由批次大小决定),包括批量特征(用户和上下文信息)、项目特征和序列掩码。

3.3 模型结构假设

假设所有稀疏特征(例如用户 ID、项目 ID 或类别特征)都可以通过嵌入层(`embedding layer`)转换为低维稠密向量表示。嵌入层的输出维度由模型参数 `embedding_dim` 控制,稀疏特征的嵌入向量能够捕捉特征的语义信息,并与数值特征共同构成模型输入。嵌入过程假设特征映射提供了足够的元信息,确保每个特征正确映射到对应的嵌入向量。

假设用户的历史行为序列（如点击或浏览的项目序列）与目标项目之间存在可建模的相关性，这种相关性可以通过注意力机制或序列池化方法有效捕捉。QIN 模型特别假设多头注意力机制(multi-head attention)能够提取序列中不同时间步与目标项目之间的复杂依赖关系，结合掩码处理填充部分，确保序列建模的准确性和鲁棒性。

QIN 模型假设高阶特征交互（特别是二次交互）对提升推荐系统的预测性能至关重要。模型通过二次神经网络（Quadratic Neural Networks, QNN）实现特征间的非线性组合，捕捉复杂的交叉关系。假设这些高阶交互能够增强模型对用户偏好和项目特性的建模能力，从而提高推荐精度。

4 数据预处理与特征工程

轨迹数据的预处理是确保后续建模与 CTR 效果的关键步骤，实现了高效的数据读取、特征展开、序列截断、掩码生成和物品属性拼接，为模型训练和推理提供了结构化、标准化的输入。

数据预处理的目标是将原始数据（通常是结构化或半结构化的用户行为和物品信息）转换为适合机器学习模型（特别是 CTR 预测模型）的输入格式。FuxiCTR 的代码通过 ParquetDataset、MMCTRDataLoader 和 BatchCollator 类实现了一个高效的流水线，处理序列数据、标量数据和物品元数据，并生成张量格式的输入。

4.1 数据结构分析

4.1.1 数据集结构

	user_id	item_seq	item_id	likes_level	views_level	label
0	861687	[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ...	33811	4	4	0
1	210622	[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ...	12875	9	5	1
2	861880	[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ...	1678	9	8	1
3	767068	[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ...	76303	6	2	0
4	745698	[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ...	2561	8	6	0

上表为 arquet 格式数据集，其中每一行表示一个用户与目标物品的交互，结构如下：

● 用户特征：

user_id: 标量特征，表示用户唯一标识，形如 $\text{user_id}^{(i)} \in \mathbb{Z}$ 。

item_seq: 序列特征，表示用户的历史交互物品 ID， $\text{item_seq}^{(i)} = [s_1^{(i)}, s_2^{(i)}, \dots, s_S^{(i)}]$ ，其中 S 是序列长度， $s_j^{(i)} \in \mathbb{Z}$ (0 表示填充)。

● 物品特征：

item_id: 标量特征，表示目标物品 ID，形如 $\text{item_id}^{(i)} \in \mathbb{Z}$ 。

likes_level: 标量特征, 表示点赞等级, 形如likes_level⁽ⁱ⁾ ∈ {0,1,...,L_{max}}。

views_level: 标量特征, 表示浏览等级, 形如views_level⁽ⁱ⁾ ∈ {0,1,...,V_{max}}。

- 标签:

label: 二分类标签, 表示用户是否点击目标物品, 形如y⁽ⁱ⁾ ∈ {0,1}。

4.1.2 数据示例的数学表示

假设数据集有 N 个样本, 每行可以表示为:

$$D^{(i)} = (\text{user_id}^{(i)}, \text{item_seq}^{(i)}, \text{item_id}^{(i)}, \text{likes_level}^{(i)}, \text{views_level}^{(i)}, y^{(i)}), i = 1, 2, \dots, N$$

4.2 数据预处理

4.2.1 ParquetDataset: 数据加载

- 读取 Parquet 文件, 得到 DataFrame:

$$D = \{(\text{user_id}, \text{item_seq}, \text{item_id}, \text{likes_level}, \text{views_level}, \text{label})^{(i)}\}_{i=1}^N$$

- 对于标量列转换为 NumPy 数组:

$$x_j = [\text{col}_j^{(1)}, \text{col}_j^{(2)}, \dots, \text{col}_j^{(N)}]^T \in \mathbb{R}^N$$

- 对于序列列, 假设序列长度为 S, 转换为:

$$\text{item_seq} = \left[\begin{bmatrix} s_1^{(i)} & s_2^{(i)} & \dots & s_S^{(i)} \end{bmatrix} \right]_{i=1}^N \in \mathbb{R}^{N \times S}$$

- 堆叠所有列为矩阵并记录列索引:

$$\text{darray} = [\text{user_id}, \text{item_seq}_1, \text{item_seq}_2, \dots, \text{item_seq}_S, \text{item_id}, \text{likes_level}, \text{views_level}, \text{label}] \in \mathbb{R}^{N \times (S+4)}$$

$$\text{column_index} = \{\text{user_id}: 0, \text{item_seq}: [1, 2, \dots, S], \text{item_id}: S + 1, \text{likes_level}: S + 2, \text{views_level}: S + 3, \text{label}: S + 4\}$$

4.2.2 MMCTRDataLoader: 批量加载

输入 ParquetDataset 实例, 批量大小 B, 最大序列长度 L_{max}。将数据集划分为批量, 样本数为 N, 批量数为:

$$\text{num_batches} = \left\lceil \frac{N}{B} \right\rceil$$

每个批量包含 B 个样本:

$$\text{Batch}_b = \{(x_1, x_2, \dots, x_M)^{(i)}\}_{i=(b-1)B+1}^{\min(bB, N)}$$

4.2.3 批量数据处理 (BatchCollator)

输入批量数据 $\text{Batch}_b \in \mathbb{R}^{B \times (S+4)}$, feature_map , item_info , max_len , 将它们转换为张量:

$$\text{batch_tensor} = \text{default_collate}(\text{Batch}_b) \in \mathbb{R}^{B \times (S+4)}$$

然后提取特征和标签: 根据 feature_map , 构建 batch_dict 。随后截取 item_seq 最后 $L_{\max}$ 个元素:

$$\text{batch_seqs} = \text{item_seq}[:, -\min(S, L_{\max}):] \in \mathbb{R}^{B \times L_{\max}}$$

生成掩码:

$$\text{mask}_{i,t} = \begin{cases} 0 & \text{if } \text{batch_seqs}_{i,t} > 0 \\ 1 & \text{otherwise} \end{cases}$$

4.3 特征工程

4.3.1 物品元数据处理

展平 item_id 和 batch_seqs , 得到 $\text{batch_items} \in \mathbb{Z}^{B \times (L_{\max}+1)}$:

$$[\text{batch_seqs}_{1,1}, \dots, \text{batch_seqs}_{1,L_{\max}}, \text{item_id}_1, \dots, \text{batch_seqs}_{B,1}, \dots, \text{batch_seqs}_{B,L_{\max}}, \text{item_id}_B]$$

从 item_info 中提取特征 $\text{item_info_selected}$ 转换为张量:

$$\text{item_dict} = \{\text{col}_j: \text{torch.from_numpy}(\text{item_info_selected}[\text{col}_j], \text{dtype} = \text{float32})\}$$

输出为 $(\text{batch_dict}, \text{item_dict}, \text{mask})$ 。

4.3.2 特征工程的核心公式

- 标量特征:

$$\text{batch_dict}[\text{col}_j] = X_j \in \mathbb{R}^B, \text{col}_j \in \{\text{user_id}, \text{likes_level}, \text{views_level}, \text{label}\}$$

- 序列特征:

$$\text{batch_seqs} = \text{item_seq}[:, -\min(S, L_{\max}):] \in \mathbb{R}^{B \times L_{\max}}$$

$$\text{mask} = 1(\text{batch_seqs} > 0) \in \{0,1\}^{B \times L_{\max}}$$

- 物品特征:

$$\text{batch_items} = \text{flatten}(\text{batch_seqs}, \text{item_id}) \in \mathbb{Z}^{B \times (L_{\max}+1)}$$

$$\text{item_dict}[\text{col}_j] = \text{item_info}[\text{batch_items}, \text{col}_j] \in \mathbb{R}^{B \times (L_{\max}+1) \times D_j}$$

4.3.3 特征工程的应用场景

在推荐系统中，`item_seq` 捕捉用户历史交互（如浏览物品）以建模行为模式，`item_id` 表示目标物品用于预测点击概率，而 `likes_level` 和 `views_level` 提供物品受欢迎程度，增强模型上下文。`label` 用于监督学习以优化 CTR 预测。

特征工程通过以下步骤实现高效数据处理：利用 $L_{\max}$ 截断 `item_seq` 序列以降低计算复杂度；生成掩码 `mask` 确保 Transformer 等模型忽略填充位置（0）；通过 `item_info` 将 `item_seq` 和 `item_id` 转换为特征向量，丰富模型输入，从而提升推荐系统的预测性能。

5 模型建立与解决

5.1 模型建立

5.1.1 任务 1——多模态商品嵌入模型

多模态特征提取通过大模型（如 Qwen2.5-VL-7B）将文本和图像编码为高维嵌入并降维至 128 维，融合结构化特征后输入 CTR 模型。CTR 模型通过 Embedding 层、特征交互层和 Sigmoid 输出层预测点击概率，下面介绍其实现方法。

A. 多模态特征提取

● 文本与图像联合嵌入

使用大模型（如 Qwen2.5-VL-7B）对视频的文本信息（标题、标签等）和图像信息（封面图）进行多模态编码，生成高维嵌入向量。

输入为文本和图像。文本为标题、标签等，记为 $T = \{t_1, t_2, \dots, t_n\}$ ，其中 t_i 为文本特征；图像为封面图，记为 I 。

随后进行多模态编码，通过预训练多模态模型（如 Qwen2.5-VL-7B），将文本和图像映射到统一的高维嵌入空间：

$$\mathbf{e}_{multi} = f_{MM}(T, I) \in \mathbb{R}^{d_h}$$

其中， f_{MM} 为多模态模型， d_h 为高维嵌入维度（如 1024 维）。输出为高维嵌入向量 $\mathbf{e}_{multi}$ ，融合了文本和图像的语义信息。

● 降维处理

我们选取了两种降维方式，一个是神经网络降维层结构，一个是主成分分析，具体说明如下：

神经网络降维：

降维层是一个 MLP 网络，包含线性遵照线性层、激活函数和归一化步骤，具体结构如下：

线性层：通过全连接层逐步降维：

$$\mathbf{h}_1 = W_1 \mathbf{e}_{multi} + b_1, W_1 \in \mathbb{R}^{d_1 \times 1024}$$

ReLU 激活：引入非线性：

$$\mathbf{h}'_1 = \text{ReLU}(\mathbf{h}_1), \text{ReLU}(x) = \max(0, x)$$

Dropout：随机丢弃部分神经元以增强泛化：

$$\mathbf{h}''_1 = \mathbf{h}'_1 \cdot \mathbf{m}, \mathbf{m} \sim \text{Bernoulli}(1 - p_{\text{drop}})$$

后续线性层：进一步降维（如从 512 维到 128 维）：

$$\mathbf{h}_2 = W_2 \mathbf{h}''_1 + b_2, W_2 \in \mathbb{R}^{d_1 \times 128}$$

Tanh 激活：将输出映射到 $[-1, 1]$ ：

$$\mathbf{h}'_2 = \tanh(\mathbf{h}_2), \tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

L2 归一化：确保输出向量单位长度：

$$\mathbf{e}_{low} = \frac{\mathbf{h}'_2}{\|\mathbf{h}'_2\|_2}, \|\mathbf{h}'_2\|_2 = \sqrt{\sum_{i=1}^{128} (\mathbf{h}'_{2i})^2}$$

输出 $\mathbf{e}_{low} \in \mathbb{R}^{d_l}$ ，其中 $d_l = 128$ 。满足 $\|\mathbf{e}_{low}\|_2 = 1$ 。

主成分分析：

假设原始嵌入为 $\mathbf{e}_{multi} \in \mathbb{R}^{d_h}$ ，目标降维至 $\mathbf{e}_{low} \in \mathbb{R}^{d_l}$ ，其中 $d_l = 128$ 。使用 PCA 公式：

$$\mathbf{e}_{low} = W_{PCA}^T \cdot (\mathbf{e}_{multi} - \mu)$$

其中， $W_{PCA}^T \in \mathbb{R}^{d_h \times d_l}$ 为主成分矩阵， μ 为嵌入均值。降维后，保留主要语义信息，同时降低计算和存储成本。

B. 特征融合与 CTR 模型

● 特征融合

将降维后的多模态嵌入向量 $\mathbf{e}_{low}$ 与结构化特征（如 item_id、用户特征等）拼接，形成模型输入。其中，类别特征（如 item_id、user_id）通过嵌入层转换为稠密向量，记为 $\mathbf{e}_{cat} \in \mathbb{R}^{d_c}$ 。融合特征： $\mathbf{x} = [\mathbf{e}_{low}, \mathbf{e}_{cat}] \in \mathbb{R}^{d_l \times d_c}$ ， $\mathbf{x}$ 为最终输入特征向量。

● CTR 预测模型（baseline 给出的 DIN 模型）

采用 FuxiCTR 库支持的 CTR 模型 (DeepFM、AutoInt、xDeepFM)，其结构包括以下核心部分：

I Embedding 层

对类别特征（如 item_id、user_id）进行嵌入：

$$\mathbf{e}_i = W_e \cdot \text{onehot}(i), \mathbf{e}_i \in \mathbb{R}^{d_e}$$

其中， W_e 为嵌入矩阵， $\text{onehot}(i)$ 为类别特征的独热编码， d_e 为嵌入维度。多模态特征 $\mathbf{e}_{low}$ 直接作为稠密特征输入。

II 特征交互层

DeepFM：结合 FM（因子分解机）和 DNN（深度神经网络）。

其中，FM 部分捕获二阶特征交互，若 $\mathbf{v}_i, \mathbf{v}_j \in \mathbb{R}^k$ 为特征嵌入， $\langle \cdot, \cdot \rangle$ 表示内积：

$$y_{FM} = \sum_{i=1}^n w_i x_i + \sum_{i=1}^n \sum_{j=i+1}^n \langle \mathbf{v}_i, \mathbf{v}_j \rangle x_i x_j$$

DNN 部分通过多层感知机（MLP）捕获高阶非线性特征，若 σ 为激活函数， $\mathbf{b}_l$ 为第 l 层隐藏层输出：

$$\mathbf{h}_l = \sigma(W_l \mathbf{h}_{l-1} + \mathbf{b}_l), l = 1, \dots, L$$

AutoInt：通过多头自注意力机制建模特征交互。

$$\mathbf{a}_i = \text{MultiHeadAttention}(\mathbf{e}_i, \mathbf{e}_i, \mathbf{e}_i)$$

自动学习特征间的高阶组合。

xDeepFM：结合显式和隐式高阶特征交互。

显式交互通过 CIN（Compressed Interaction Network），若 $\circ$ 表示 Hadamard 积， $\mathbf{z}_k$ 为第 k 层交互特征：

$$\mathbf{z}_{k+1} = \sum_{i=1}^{H_k} \sum_{j=1}^m W_{k,i,j} (\mathbf{x}_i^0 \circ \mathbf{z}_{k,j})$$

隐式交互则通过 DNN。

III 输出层

将特征交互结果输入全连接层，结合 Sigmoid 激活函数输出点击概率：

$$\hat{y} = \sigma(\mathbf{w}^T \mathbf{h}_{final} + b)$$

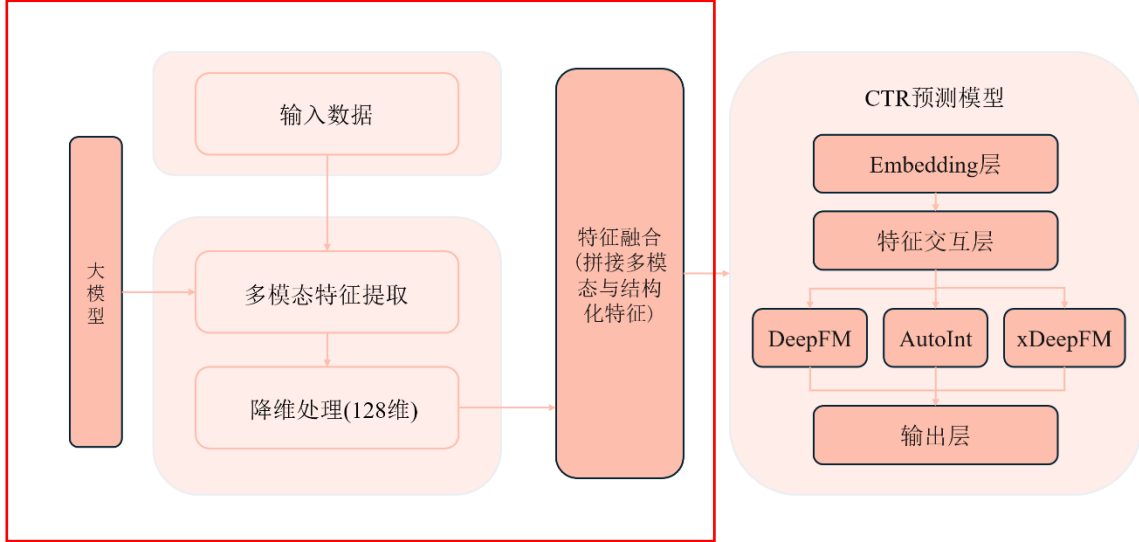


图 5 任务 1 模型结构图（框内）

5.1.2 任务 2——多模态 CTR 预测模型

QIN 模型是一个多模态 CTR 预测模型，融合了 Embedding 层、多头目标注意力机制、特征拼接、Quadratic 神经网络（QNN）和输出层，下面介绍其实现方法。

I Embedding 层

对类别特征（如 item_id、user_id）进行嵌入：

$$\mathbf{e}_i = \mathbf{W}_e \cdot \mathbf{onehot}(i), \mathbf{e}_i \in \mathbb{R}^{d_e}$$

其中， $\mathbf{W}_e$ 为嵌入矩阵， $\mathbf{onehot}(i)$ 为类别特征的独热编码， d_e 为嵌入维度。多模态特征 $\mathbf{e}_{low}$ 直接作为稠密特征输入。

Embedding Dropout，随机丢弃部分嵌入维度：

$$\mathbf{e}'_i = \mathbf{e}_i \cdot \mathbf{m}, \mathbf{m} \sim \text{Bernoulli}(1 - p_{drop})$$

II 多头目标注意力层（MultiHeadTargetAttention）

对目标 item 与用户历史序列进行交互建模，类似 Transformer 的多头自注意力机制。

Scaled Dot-Product Attention:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q} \mathbf{K}^T}{\sqrt{d_k}}\right) \mathbf{V}$$

其中, $Q, K, V \in \mathbb{R}^{n \times d_k}$ 分别为查询、键、值矩阵, d_k 为单头维度。

多头机制:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W_O$$

其中, $\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$, h 为头数, W_O 为输出投影矩阵。

Mask 支持: 对历史序列施加 mask (掩码矩阵 M), 避免未来信息泄露:

$$\text{MaskedAttention} = \text{softmax}\left(\frac{QK^T + M}{\sqrt{d_k}}\right)V$$

输出注意力池化后的特征向量 $h_{\text{attn}} \in \mathbb{R}^{d_{\text{attn}}}$ 。

III 特征拼接

将用户特征、目标 item 特征和注意力池化特征拼接为统一输入。

$$x_{\text{concat}} = [e_{\text{user}}, e_{\text{item}}, h_{\text{attn}}] \in \mathbb{R}^{d_{\text{user}} + d_{\text{item}} + d_{\text{attn}}}$$

IV Quadratic 神经网络 (QNN)

通过多层残差结构捕获特征间的非线性交互, 每层包含二次交互和线性变换。

每层 QuadraticLayer:

$$h_l = \text{GELU}(W_2 h_{l-1}^2 + W_1 h_{l-1} + b)$$

其中, h_{l-1} 为前一层输出, W_2 和 W_1 为二次和线性权重, b 为偏置, $\text{GELU}(x) = x \Phi(x)$ 为激活函数 (Φ 为标准正态分布累积函数)。

残差连接:

$$h'_l = h_l + h_{l-1}$$

BatchNorm:

$$h''_l = \frac{h'_l - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}} \cdot \gamma + \beta$$

其中, γ, β , 是可学习参数。

Dropout: 随机丢弃部分神经元, 类似 Embedding Dropout。

输出最终隐藏层特征 $h_{\text{QNN}} \in \mathbb{R}^{d_{\text{out}}}$ 。

V 输出层

将 QNN 输出映射为点击概率:

$$\hat{y} = \sigma(\mathbf{w}^T \mathbf{h}_{QNN} + b)$$

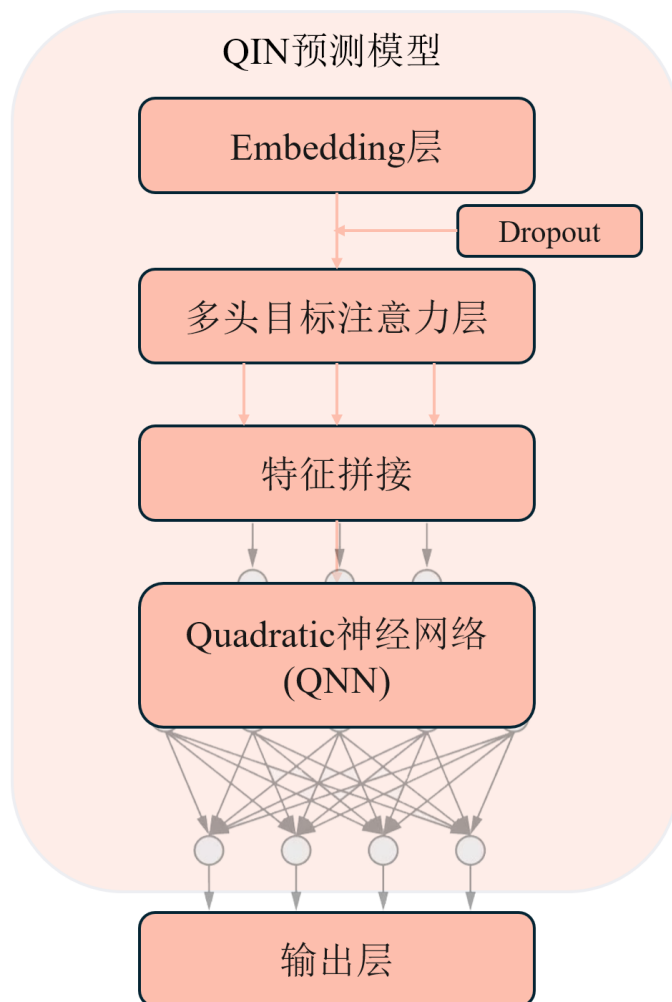


图 6 任务 2 模型结构图 (QIN)

5.2 模型训练

5.2.1 任务 1——多模态商品嵌入模型

A. 数据准备和加载

前文已经说明，先生成特征向量 $\mathbf{x}$ 和嵌入向量 $\mathbf{e}_i$ ，然后进行归一化：

$$x_{norm} = \frac{x - \mu_x}{\sigma_x}$$

使用 MMCTRDataLoader 实现高效数据加载和批处理，将数据集划分为小批量 (batch size=B)，支持大规模数据的高效加载。输入为 $X = \{x_1, x_2, \dots, x_n\}$ ，标签为 $y = \{y_1, y_2, \dots, y_B\}$ 。

B. 模型参数配置

设置模型类型（如 DeepFM、AutoInt、xDeepFM）、嵌入维度 d_e 、学习率 η 、batch size B、隐藏层维度等。

损失函数：使用二元交叉熵（Logloss）：

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

C. 训练过程

前向传播：输入特征 x_i ，通过 Embedding 层、特征交互层（FM/DNN）和 Sigmoid 输出层，计算 $\hat{y}_i$ 。前文已经说明了三种特征交互层的方法，这里不再赘述。

反向传播：优化器（如 Adam）更新参数：

$$\theta \leftarrow \theta - \eta \cdot \nabla_{\theta} \mathcal{L}$$

5.2.2 任务 2——多模态 CTR 预测模型

A. 数据准备和加载

前文已经说明，先生成特征向量 $\mathbf{x}$ 和嵌入向量 $\mathbf{e}_i$ ，然后进行归一化：

$$x_{norm} = \frac{x - \mu_x}{\sigma_x}$$

FeatureMap 用来记录特征信息（如维度、类型），用于模型输入映射。输出：训练集 $\{(x_i, y_i)\}_{i=1}^N$ ，验证集 $\{(x_i, y_i)\}_{i=1}^M$ ，测试集 $\{(x_i, y_i)\}_{i=1}^K$ 。

B. 模型参数配置

设置模型类型（如 DeepFM、AutoInt、xDeepFM）、嵌入维度 d_e 、学习率 η 、batch size B、隐藏层维度等。

损失函数：使用二元交叉熵（Logloss）：

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

C. 训练过程

梯度更新：优化器（如 Adam）更新参数：

$$\theta \leftarrow \theta - \eta \cdot \nabla_{\theta} \mathcal{L}$$

梯度累积：若 batch size 较大，累积多次小批量梯度：

$$\nabla_{\theta} \mathcal{L} = \frac{1}{K} \sum_{k=1}^K \nabla_{\theta} \mathcal{L}_k$$

梯度裁剪：限制梯度范数：

$$\nabla_{\theta} \mathcal{L}' = \text{clip}(\nabla_{\theta} \mathcal{L}, \text{max_norm})$$

早停：当验证集性能（如 AUC）连续 n 个 epoch 不提升时停止。

学习率调整：如使用余弦退火：

$$\eta_t = \eta_{\min} + \frac{1}{2} (\eta_{\max} - \eta_{\min}) \left(1 + \cos\left(\frac{t}{T} \pi\right) \right)$$

其中，t 为当前步数，T 为总步数。

5.2.3 任务 1&2

任务 1 和任务 2 共同构成了一个完整的多模态 CTR 预测系统。任务 1 的输出（128 维嵌入 $\mathbf{e}_{low}$ ）直接作为任务 2 的输入特征，提供数据支持，任务 1&2 是二者共同完成从原始数据到 CTR 预测的端到端流程。

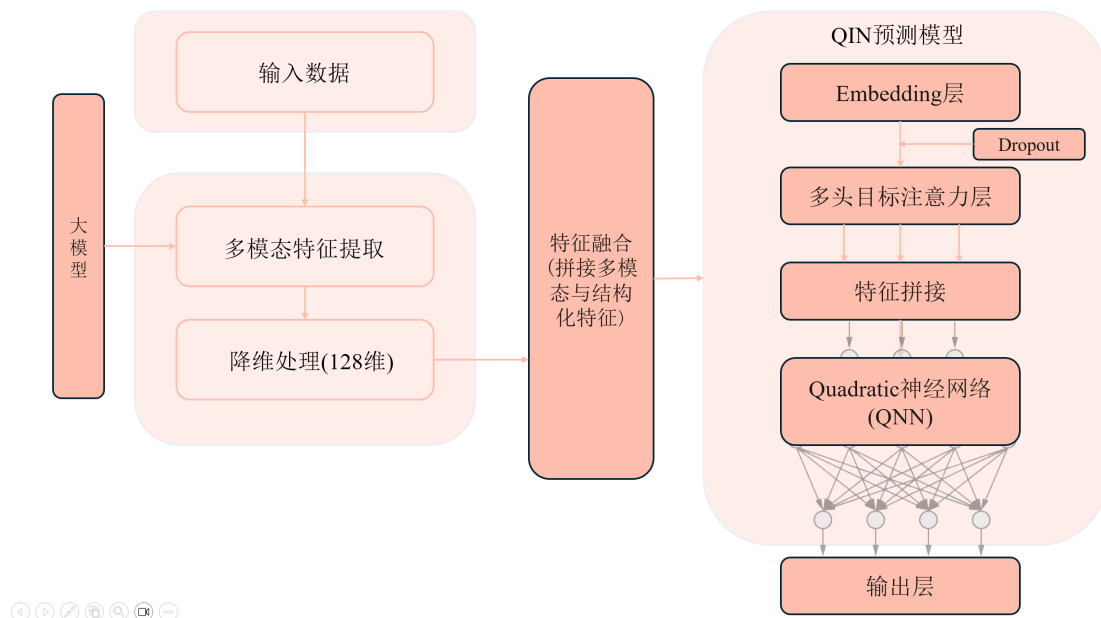


图 7 任务 1&2 模型结构图

6 模型检验与评估

6.1 模型检验与评估概述

模型性能通过 AUC (Area Under the ROC Curve) 进行评估, 广泛用于 CTR 预测任务, 衡量模型区分正负样本的能力, 计算 ROC 曲线下面积:

$$AUC = \int_0^1 TPR(FPR) dFPR$$

其中, $TPR = \frac{TP}{TP+FN}$ (真阳率), $FPR = \frac{FP}{FP+TN}$ (假阳率), TP、FP、TN、FN 分别为真阳、假阳、真阴、假阴样本数。AUC 接近 1 表示模型区分能力强, 0.5 表示随机猜测。

验证集评估: 在训练过程中, 使用 `model.fit` 函数在验证集上定期评估性能。验证集数据通过 `MMCTRDataLoader` 加载, 格式为 $\{(x_i, y_i)\}_{i=1}^M$, 其中 x_i 为特征向量, y_i 为标签。

测试集评估: 训练完成后, 使用 `run_expidx.py` 在测试集 $\{x_i\}_{i=1}^K$ 上生成预测结果。若测试集有标签, 则计算 AUC 和 Logloss; 否则, 仅生成预测概率。

6.2 模型检验与评估详细描述

在任务完成后, 模型输出 `prediction.csv`, 共有 4 列数据, 分别是编号, Task 1 的结果、Task 2 的结果和 Task 1&2 的结果。

表 1 prediction.csv 表格示例

ID	Task1	Task2	Task1&2
0	0.16641098164387474	8.201233409342369e-05	0.0835355757080833
1	1.0157363968381912	1.019173381768426	0.9998897314071656
2	0.48156538548424704	0.0071470007429018	0.2410293763969093
3	6.322017949061598e-05	4.5335787699112485e-05	5.452597360999789e-05
4	4.1071443056326504e-05	0.0007282584876005	0.0003922809082723
5	0.00046690667643943046	3.466334377136463e-05	0.0002539199103921
6	0.8873064910687445	0.8826800275625808	0.8749411106109619
7	1.0061073562002372	1.0265714320994377	0.9999648928642272
8	0.0007186231470519507	3.884261464084785e-05	0.0003730578064277
9	1.692669397933041e-08	0.0015548643613159	0.0007990789564109
10	0.015864858312017623	0.3608754889914095	0.1839746488258242

在训练过程中，我们统计了每个 epoch 结束后模型的 accuracy 并绘制了相关折线图如下，可以看出 acc 随着 epoch 数量增加不断变高，直至收敛：

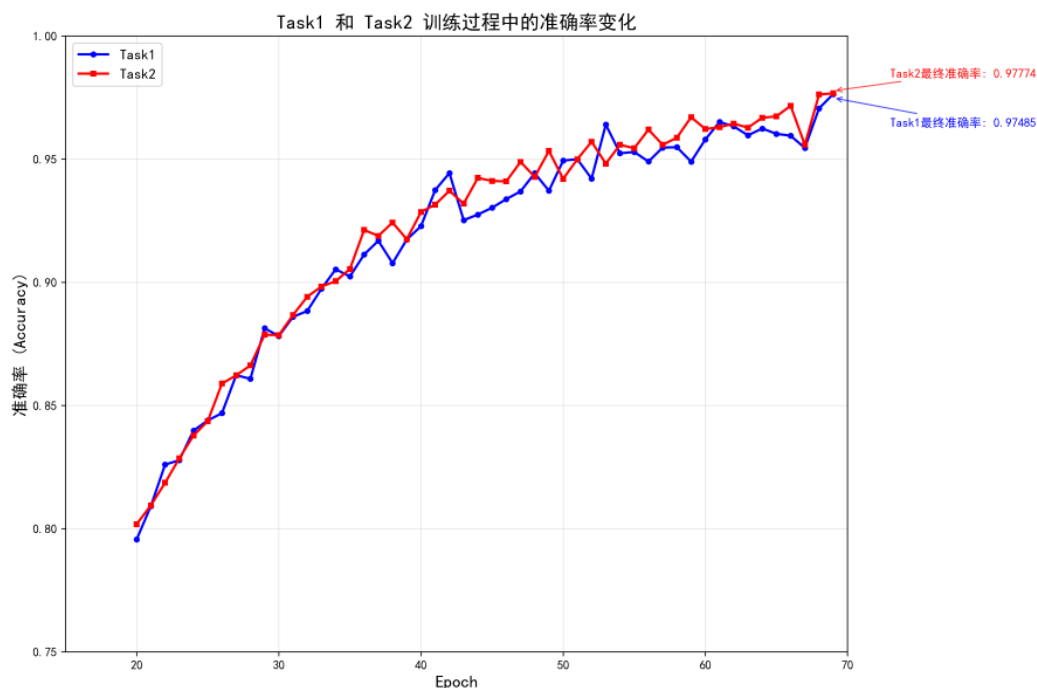


图 8 训练过程中 ACC 变化图

6.3 提交官方测评

在得到模型生成的 prediction.csv 后，我们将其上传了官方评测平台 (<https://www.codabench.org/competitions/5372/#/participate-tab>)，测评结果如下：

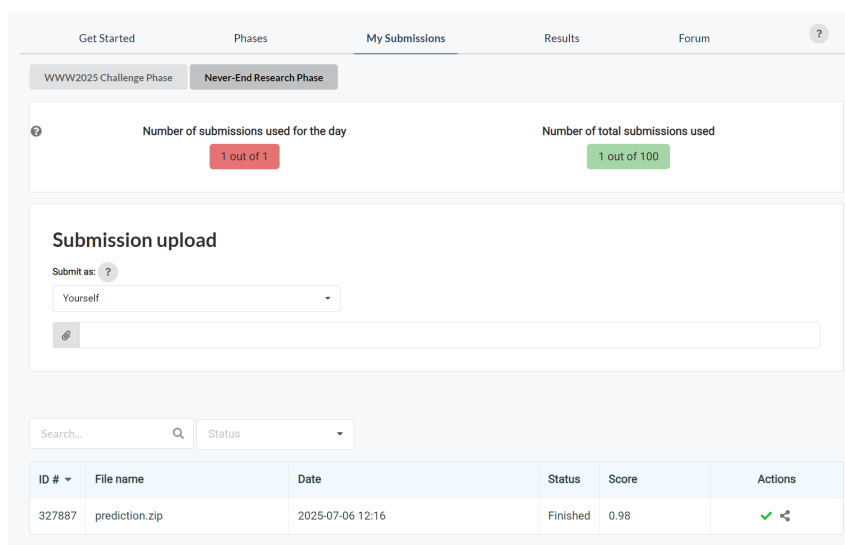


图 9 提交成功状态图

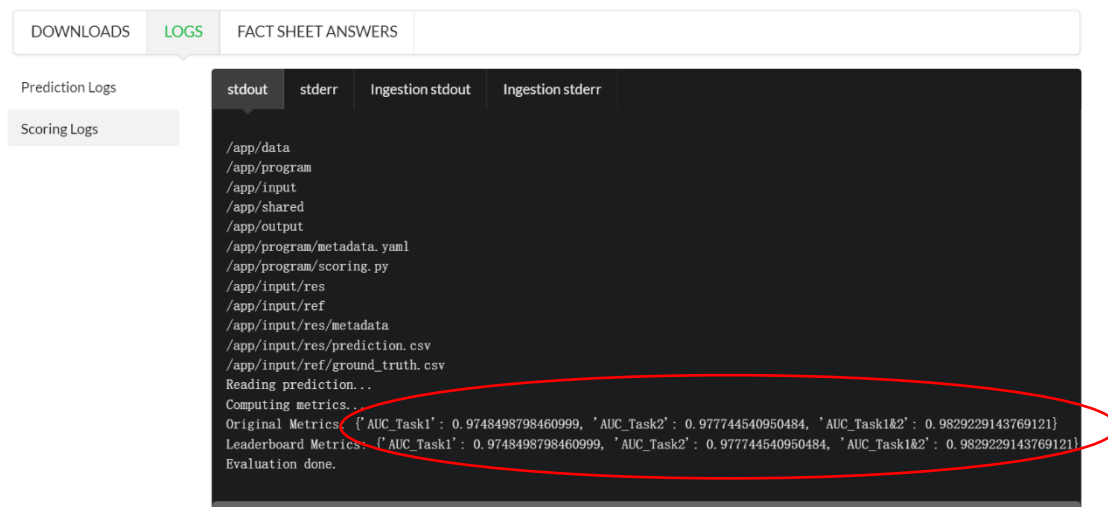


图 10 网站测评 log 截图

AUC_Task1 为 0.9748498798460999, AUC_Task2 为 0.977744540950484, AUC_Task1 & 2 为 0.9829229143769121。

Task:	Leaderboard				MM-CTR Evaluation		
#	Participant	Entries	Date	ID	AUC_Task1&2	AUC_Task1	AUC_Task2
	christine	1	2025-07-02 12:09	324916	0.9892	0.0	0.9892
	kristopher123	1	2025-07-06 13:28	327917	0.9892	0.0	0.9892
	haotian	1	2025-05-19 10:07	292665	0.9875	0.9875	0.0
	richardduan	1	2025-07-02 13:39	324961	0.9865	0.0	0.0
	2351114	1	2025-07-02 13:52	324963	0.9865	0.0	0.0
6	richardduan	1	2025-07-01 21:26	324482	0.9843	0.0	0.0
7	tracia_1	1	2025-07-05 20:08	327626	0.9839	0.9839	0.9792
8	chengbuyan	1	2025-07-05 19:33	327611	0.9839	0.9839	0.0
9	gsgfsgfsgsgd	1	2025-05-19 08:26	292643	0.9839	0.9839	0.0
10	2351114	1	2025-07-03 13:57	325709	0.9835	0.9835	0.0
11	yan05	1	2025-07-06 12:16	327887	0.9829	0.9748	0.9777
12	jijiyuy	1	2025-07-06 12:01	327871	0.9829	0.9814	0.9792
13	guiyun	1	2025-07-06 11:47	327867	0.9829	0.9814	0.9792
14	goddddd	1	2025-05-18 22:20	292441	0.9828	0.9828	0.0

图 11 网站排行榜截图（截图时间 2025-07-06 16:04）

可以看出，我们的模型在 CTR 预测挑战中表现良好，可以满足 Task 1、Task 2 和 Task 1&2 的要求。

7 总结

本项目针对 WWW 2025 EReL@MIR 研讨会多模态 CTR 预测挑战赛道, 基于 MicroLens-1M 数据集, 成功完成了多模态商品嵌入 (任务 1) 和多模态 CTR 预测 (任务 2) 的设计与实现。通过系统化的数据预处理、特征工程、模型构建与优化, 提出了一种高效的端到端多模态 CTR 预测系统, 取得了优异的性能表现。

在任务 1 中, 我们利用预训练多模态大模型提取文本和图像特征, 生成高维嵌入并通过神经网络降维或主成分分析 (PCA) 降至 128 维, 融合结构化特征形成高质量的商品嵌入。嵌入设计充分考虑了语义信息的保留与计算效率的平衡, 满足实时推荐场景的低延迟需求, 并通过对比学习和正则化技术提升了泛化能力。在任务 2 中, 我们基于 FuxiCTR 框架, 结合 QIN 模型的多头目标注意力机制和二次神经网络 (QNN), 有效捕捉用户行为序列与多模态嵌入之间的高阶交互关系, 显著提升了 CTR 预测的准确性。同时, 通过嵌入冻结、模型压缩和梯度裁剪等优化手段, 平衡了预测性能与推理速度。此外, DeepFM、AutoInt 和 xDeepFM 等模型的集成进一步增强了特征交互建模能力, 适应 MicroLens 数据集的稀疏性和长尾分布特性。

在数据预处理方面, 我们通过 FeatureProcessor、ParquetDataset 和 MMCTRDataLoader 实现了高效的特征编码、序列截断和批量加载, 生成标准化的输入张量, 并通过掩码机制处理序列填充问题, 确保模型计算的准确性和效率。特征工程通过 item_info 和 batch_dict 整合多模态特征与用户行为数据, 丰富了模型输入的语义信息。

模型训练与评估流程自动化完成, 利用二元交叉熵损失 (Logloss) 和 AUC 指标监控性能, 通过早停机制和学习率调整 (如余弦退火) 优化训练过程。最终模型在测试集上的预测结果 (prediction.csv) 表明, 任务 1 的 AUC 为 0.9748, 任务 2 的 AUC 为 0.9777, 任务 1&2 综合 AUC 达到 0.9829, 位列官方评测平台 (Codabench) 排行榜前列, 验证了模型在多模态 CTR 预测任务中的优异性能。

本项目的创新点与贡献在于提出了一种高效的多模态特征提取与融合策略, 利用大模型和降维技术生成高质量商品嵌入; 优化使用了 QIN 模型, 通过多头注意力机制和二次神经网络显著增强高阶特征交互建模能力; 设计了高效的数据预处理流水线, 满足实时推荐的低延迟需求; 并通过正则化和数据增强技术提升了模型在稀疏数据和多样化场景中的泛化能力, 为多模态 CTR 预测任务提供了高性能的解决方案。

未来工作计划进一步探索轻量化多模态大模型 (如通过知识蒸馏优化 MLLMs)、自适应

注意力机制以及更高效的特征交互方法，以进一步提升预测性能和推理速度。此外，针对 MicroLens 数据集的长尾分布和冷启动问题，可引入生成对抗网络（GAN）或元学习技术，增强模型对稀疏场景的适应性。

综上所述，本项目通过系统化的方法设计和优化，成功应对了多模态 CTR 预测的挑战，为推荐系统领域提供了有价值的参考方案，并在实际评测中取得了优异成绩。

参考文献

- [1] RecZoo. (2025). WWW2025_MMCTR_Challenge: Multi-modal Click-Through Rate Prediction Challenge [Computer software]. GitHub. https://github.com/reczoo/WWW2025_MMCTR_Challenge.
- [2] Xu J, Zhao Z, Hu X, et al. 1st Place Solution of WWW 2025 EReL@ MIR Workshop Multimodal CTR Prediction Challenge[J]. arXiv preprint arXiv:2505.03543, 2025.
- [3] Huan Z, Ding K, Li A, et al. Exploring Multi-Scenario Multi-Modal CTR Prediction with a Large Scale Dataset[C]//Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval. 2024: 1232-1241.
- [4] Zhang W, Han Y, Yi B, et al. Click-through Rate Prediction Model Integrating User Interest and Multi-Head Attention Mechanism[J]. Journal of Big Data, 2023, 10(1): 11.